

Inventory Reservation System

Technical Summary & Reviewer Guide (Run and Test)

Candidate	[Your name]
Position	[Role you are applying for]
Submitted	[Date]
Repository / archive	[Link or file name]
Stack	.NET 9 · ASP.NET Core · EF Core 9 (Pomelo) · MySQL 8 · JWT · Serilog · xUnit · Docker

This guide gives the reviewer everything needed to understand, run and verify the solution in about 15 minutes: a technical summary of the design decisions, step-by-step instructions to start the system with Docker, and how to run the full automated test suite, including the 500-request concurrency tests.

1. What the system does

A REST API for reserving stock. A user reserves units of a product for **2 minutes**, then **confirms** (purchase) or **cancels**. Unconfirmed reservations **expire automatically** and their stock is released.

Core guarantees:

- Stock is **never oversold**: **ReservedQuantity** + **SoldQuantity** can never exceed **TotalQuantity**, under any concurrency and on any number of API instances.
- Each reservation makes **exactly one** terminal transition (Confirmed, Cancelled or Expired), so stock is never sold twice or released twice.
- Security (JWT, roles, ownership), structured logging with correlation IDs, rate limiting and consistent, non-leaking error responses.

Concurrency Verification

Stock: 1
Requests: 500
Success: 1
Rejected: 499
Overselling: 0

500 reservations released simultaneously against real MySQL, each with its own DbContext and connection, across two independent service containers (simulating two API instances). With stock 100: **100 succeed, 400 rejected**. Through the full HTTP stack with 500 different users and production rate limits: **100 × 201, 400 × 409, 0 × 429**. Repeated many times with zero failures.

The API is stateless and horizontally scalable: correctness is enforced by MySQL (atomic conditional updates inside transactions), not by process-local locks.

2. Architecture

Clean Architecture in four projects; dependencies point inwards and the domain has no framework references.

Project	Responsibility	Depends on
Inventory.Domain	Entities (Product , Reservation , User), invariants, reservation state machine.	nothing
Inventory.Application	Use cases, validation, DTOs, ports (IReservationRepository , ICurrentUser , IPasswordHasher , IJwtTokenGenerator).	Domain
Inventory.Infrastructure	EF Core + MySQL: DbContext, configurations, migrations, repositories with the atomic SQL; PBKDF2 hashing; JWT issuing.	Application, Domain
Inventory.Api	Thin controllers, JWT validation, authorization, ProblemDetails, Serilog, correlation IDs, rate limiting, Swagger, health checks, expiry worker.	Application, Infrastructure
Inventory.UnitTests	Domain and application logic, no I/O.	Domain, Application

Project	Responsibility	Depends on
Inventory.IntegrationTests	Real ASP.NET Core pipeline; real MySQL when configured.	Api, Infrastructure

Request pipeline

Correlation ID → Serilog request log → exception handler (ProblemDetails) → JWT authentication → rate limiter → authorization → controller → application service → repository → MySQL. The **ReservationExpiryWorker** (a BackgroundService) runs on every instance.

Database

Table	Key columns	Constraints & indexes
Products	Id, Sku, Name, TotalQuantity, ReservedQuantity, SoldQuantity, CreatedAtUtc, UpdatedAtUtc	Unique SKU; CHECK quantities ≥ 0 and Reserved + Sold ≤ Total
Reservations	Id, ProductId (FK), UserId, Quantity, Status, CreatedAtUtc, ExpiresAtUtc, ConfirmedAtUtc, CancelledAtUtc	CHECK Quantity > 0; indexes on ProductId, Status, ExpiresAtUtc, (Status, ExpiresAtUtc)
Users	Id, Username, PasswordHash, Role	Unique username (case-insensitive)

AvailableQuantity = **Total** - **Reserved** - **Sold** is computed, never stored. Timestamps are UTC **datetime(6)**. Schema is managed by EF Core migrations (**InitialCreate**, **AddUsers**).

3. Key design decisions

3.1 Concurrency: the database is the arbiter

There is no “read stock → check in C# → write later” anywhere. Each state change is one guarded SQL statement that InnoDB evaluates under the row lock; competing requests queue on the row and re-check the condition against the latest committed data, so exactly one can win.

Reserve: atomic conditional update (EF Core ExecuteUpdateAsync, parameterized)

```
UPDATE Products
SET     ReservedQuantity = ReservedQuantity + @quantity, UpdatedAtUtc = @now
WHERE  Id = @productId
      AND TotalQuantity - SoldQuantity - ReservedQuantity >= @quantity;
-- 1 row -> stock held, INSERT the Active reservation (same transaction)
-- 0 rows -> rollback; 404 PRODUCT_NOT_FOUND or 409 INSUFFICIENT_STOCK
```

Operation	Guard	Then, in the same transaction
Reserve	Total - Sold - Reserved >= @q	INSERT Active reservation (rollback if the insert fails)
Confirm	Status = 'Active' AND ExpiresAtUtc > @now	Reserved -= q, Sold += q
Cancel	Status = 'Active'	Reserved -= q
Expire	Status = 'Active' AND ExpiresAtUtc <= @now	Reserved -= q

3.2 Why a C# lock was not used

- A `lock/SemaphoreSlim` only works inside one process; with two API instances it no longer protects anything.
- It serializes all requests through one thread, while MySQL already locks exactly the conflicting rows.
- It cannot coordinate with other writers (the expiry worker on another instance, other services, manual SQL).

3.3 Two-minute expiry

- `ExpiresAtUtc = CreatedAtUtc + 2 min`. Confirm is refused at/after expiry in both the domain and the SQL guard.
- `ReservationExpiryWorker` sweeps every 15 s (configurable) using the `(Status, ExpiresAtUtc)` index. All instances may run it; the conditional transition prevents double releases (tested with 8 concurrent sweepers).
- Time is injected through `TimeProvider`, so tests move the clock instead of waiting.

3.4 Security

- JWT (HMAC-SHA256) with `sub, username, role`; issuer, audience, lifetime and algorithm validated. The signing key comes only from environment/secret store.
- Passwords: PBKDF2-SHA256, 600,000 iterations, random salt, constant-time compare; unknown users cost the same time (no username enumeration).
- Authenticated by default (fallback policy). `POST /products` is Admin-only. Users can confirm/cancel only their own reservations.

3.5 Observability, errors, rate limiting

- Serilog structured JSON. Each request logs `CorrelationId`, method, path, status, elapsed ms (+ `UserId/ProductId/ReservationId`). Headers, bodies, tokens and passwords are never logged (tested).
- `X-Correlation-ID` is accepted (validated) or generated, returned in the response and included in every error.
- Errors are RFC 9457 `application/problem+json` with stable `errorCode` (e.g. `INSUFFICIENT_STOCK`, `INVALID_RESERVATION_STATE`) and `correlationId`; unexpected errors return a generic 500 with no internals.
- Built-in rate limiter per client: general 600/min, login 10/min, reservations burst 60 (+30/10 s). Rate limiting is not concurrency control; the database still guarantees correctness.

4. API endpoints

Method & route	Access	Success	Typical errors
POST /api/v1/auth/login	anonymous	200 JWT	400, 401, 429
POST /api/v1/products	Admin	201 + Location	400, 401, 403, 409 DUPLICATE_SKU
GET /api/v1/products/{id}	user	200	401, 404

Method & route	Access	Success	Typical errors
POST /api/v1/reservations	user	201	400, 404, 409 INSUFFICIENT_STOCK, 429
POST /api/v1/reservations/{id}/confirm	owner / Admin	200	403, 404, 409
POST /api/v1/reservations/{id}/cancel	owner / Admin	200	403, 404, 409
GET /health · GET /health/ready	anonymous	200	503 (ready: database down)

5. How to run

5.1 Prerequisites

- Docker Desktop (or Docker Engine with Compose v2), running.
- .NET 9 SDK, only needed to run the automated tests or the API outside Docker.
- Free ports 8080 (API) and 3306 (MySQL). If 3306 is taken, set `MYSQL_PORT` (e.g. 3307) in `.env`.

5.2 Start the system (Docker, recommended)

1. Open a terminal in the solution folder (the one containing `docker-compose.yml`).
2. Create the environment file from the template and set real values. Generate the JWT key with `openssl rand -base64 48` (or any random string of 32+ characters).

```
cp .env.example .env # Windows PowerShell: Copy-Item .env.example .env

# edit .env:
MYSQL_ROOT_PASSWORD=<strong password>
MYSQL_PASSWORD=<strong password>
JWT_SIGNING_KEY=<random secret, at least 32 characters>
DEMO_ADMIN_USERNAME=admin
DEMO_ADMIN_PASSWORD=<admin password>
```

3. Build and start both services:

```
docker compose up --build
```

4. Wait until both containers are healthy (about 30–60 s on first build). The API waits for MySQL, applies the EF Core migrations, seeds the admin user and then reports healthy.

```
docker compose ps # both services: (healthy)
```

URL	Purpose
http://localhost:8080/swagger	Interactive API documentation
http://localhost:8080/health	Liveness (always 200 while running)
http://localhost:8080/health/ready	Readiness (200 only when MySQL is reachable)

5.3 Try it in Swagger

Open <http://localhost:8080/swagger> and follow the **step-by-step walkthrough in section 6**, which covers login, authorization, the full reservation lifecycle and the error cases, with screenshots.

5.4 The same flow with curl (optional)

```
TOKEN=$(curl -s -X POST localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"<DEMO_ADMIN_PASSWORD>"}' \
| sed -E 's/.*"accessToken":"([^\"]+)"\.*/\1/')

curl -X POST localhost:8080/api/v1/products \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"sku":"IPHONE-001","name":"iPhone","totalQuantity":10}'
```

5.5 Stop / reset

```
docker compose down          # stop; data is kept in the mysql-data volume
docker compose down -v       # stop and delete all data
```

6. Sample Swagger test (step by step)

This walkthrough exercises the complete reservation lifecycle and the main error cases through Swagger UI. It takes about **5 minutes** (plus a 2-minute wait for the expiry step). The screenshots were captured from the running Docker stack; IDs and timestamps will differ on your machine.

Before you start: the stack is running (`docker compose up --build`) and you know the admin credentials from `.env` (`DEMO_ADMIN_USERNAME` / `DEMO_ADMIN_PASSWORD`).

Swagger basics	
Expand an endpoint	Click its coloured row (e.g. the green POST bar).
Enable editing	Click Try it out (top right of the expanded endpoint).
Send the request	Edit the request body or the <code>id</code> field, then click Execute .
Read the result	Scroll to Server response : status code and response body.

Step 0. Open Swagger UI

Browse to <http://localhost:8080/swagger>. Protected endpoints show a padlock; login and health do not.

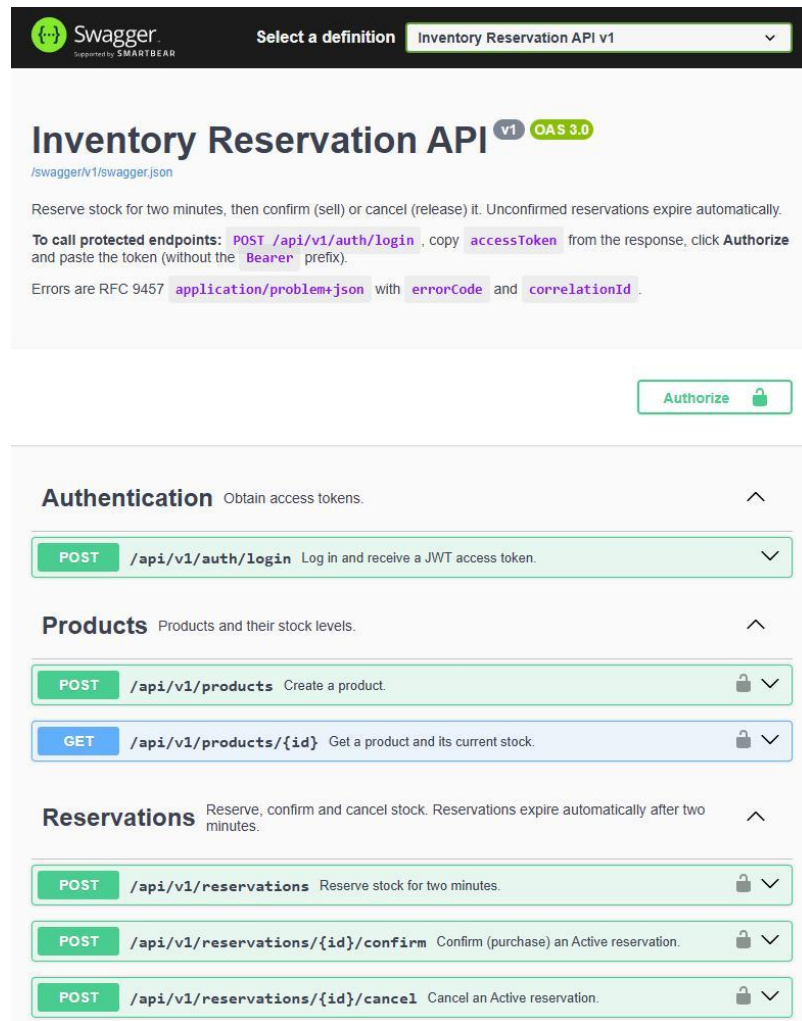


Figure 1: Swagger UI with the Authorize button and padlocked endpoints

Step 1. Log in and get a JWT

Endpoint: **POST /api/v1/auth/login**

- Expand the endpoint → **Try it out** → replace the body with your credentials → **Execute**.

```
{
  "username": "admin",
  "password": "<DEMO_ADMIN_PASSWORD from .env>"
}
```

Expected: **200 OK** the body contains **accessToken**, **tokenType**: "Bearer" and **expiresAtUtc** (60 minutes).

- Copy the value of accessToken** (without the quotes). You need it in Step 2.



Figure 2: Login response (token shortened in this image)

Step 2. Authorize Swagger with the token

- Scroll to the top and click **Authorize**.
- Paste the token into **Value**. Do not type the word Bearer; Swagger adds it.
- Click **Authorize**, then **Close**. The padlocks now appear closed.

Expected: **Authorized** the dialog shows "Authorized" and the value is masked.

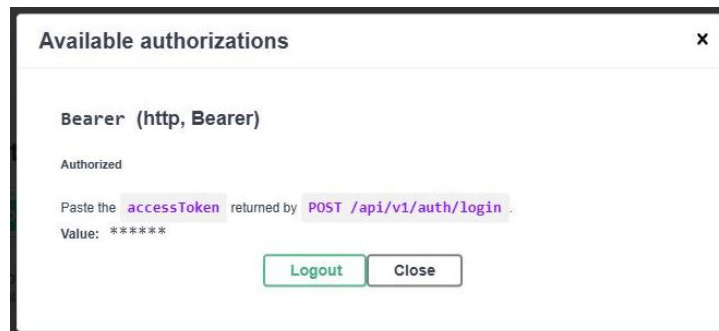


Figure 3: Authorize dialog after a successful authorization

Step 3. Create a product (Admin only)

Endpoint: POST /api/v1/products

```
{
  "sku": "DEMO-PHONE-001",
  "name": "Demo Phone",
  "totalQuantity": 3
}
```

Expected: **201 Created** reservedQuantity: 0, soldQuantity: 0, availableQuantity: 3. A Location header points to the new product.

- **Copy the product id** for the next steps. (Running it twice returns **409 DUPLICATE_SKU**; use another SKU.)



Figure 4: Product created

Step 4. Reserve stock

Endpoint: POST /api/v1/reservations

```
{
  "productId": "<product id from Step 3>",
  "quantity": 2
}
```

Expected: **201 Created** status: "Active" and expiresAtUtc exactly 2 minutes after createdAtUtc.

- **Copy the reservationId.** Complete Step 6 within 2 minutes, otherwise the reservation expires.

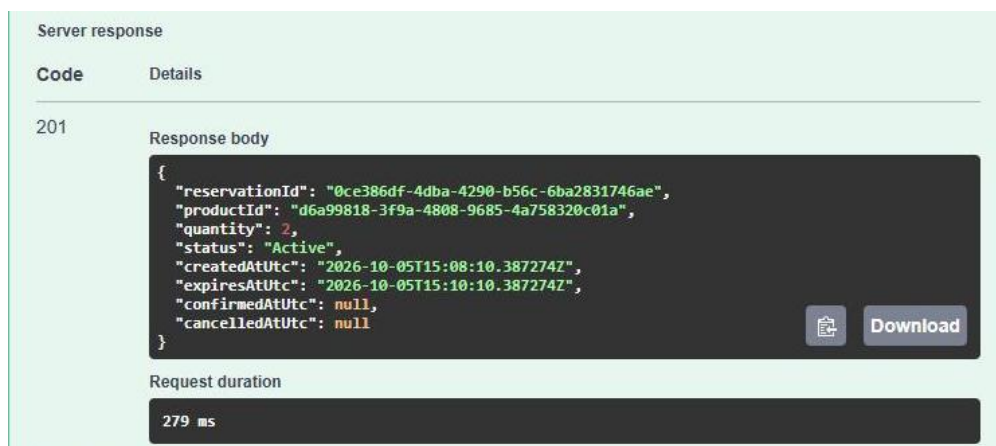


Figure 5: Active reservation holding 2 units

Step 5. Try to oversell (insufficient stock)

Endpoint: POST /api/v1/reservations (same body, quantity 2)

Only 1 unit is still available (3 total – 2 reserved), so the database rejects the hold atomically.

Expected: **409 Conflict** ProblemDetails with title: "Insufficient inventory" and errorCode: "INSUFFICIENT_STOCK". Nothing is reserved.



Server response

Code	Details
409	Error: Conflict

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "Insufficient inventory",
  "status": 409,
  "detail": "Product 'd6a99818-3f9a-4808-9685-4a758320c01a' does not have 2 unit(s) available.",
  "instance": "/api/v1/reservations",
  "errorCode": "INSUFFICIENT_STOCK",
  "correlationId": "60c3f877ee66443390cbf85d77630d2c"
}
```

Download

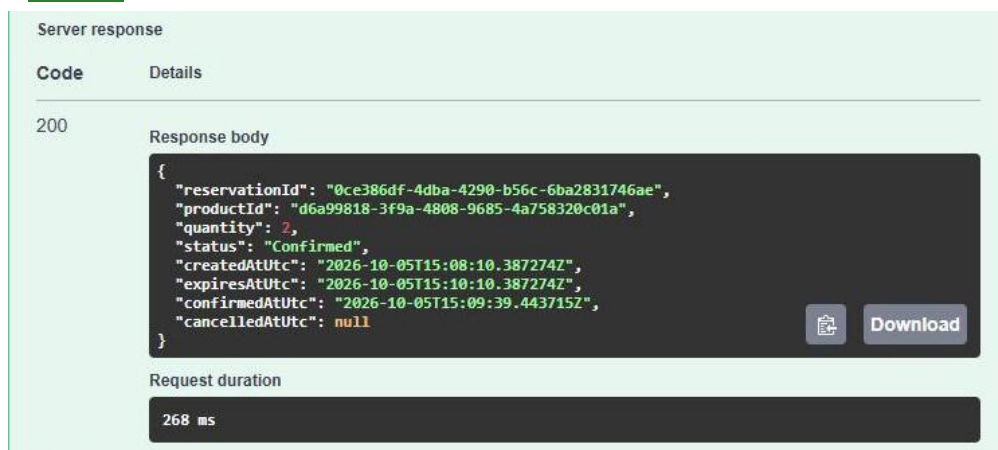
Figure 6: Overselling is rejected

Step 6. Confirm the reservation (purchase)

Endpoint: POST /api/v1/reservations/{id}/confirm

- Paste the **reservationId** from Step 4 into the **id** field → **Execute** (no body).

Expected: **200 OK** status: "Confirmed" with **confirmedAtUtc** set. The 2 reserved units are now sold.



Server response

Code	Details
200	

Response body

```
{
  "reservationId": "0ce386df-4dba-4290-b56c-6ba2831746ae",
  "productId": "d6a99818-3f9a-4808-9685-4a758320c01a",
  "quantity": 2,
  "status": "Confirmed",
  "createdAtUtc": "2026-10-05T15:08:10.387274Z",
  "expiresAtUtc": "2026-10-05T15:10:10.387274Z",
  "confirmedAtUtc": "2026-10-05T15:09:39.443715Z",
  "cancelledAtUtc": null
}
```

Request duration

268 ms

Download

Figure 7: Reservation confirmed

Step 7. Confirm the same reservation again

Endpoint: POST /api/v1/reservations/{id}/confirm (same id)

Expected: **409 Conflict** errorCode: "INVALID_RESERVATION_STATE", because a Confirmed reservation can never transition again (no double sale).



Server response

Code	Details
409	Error: Conflict

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "Invalid reservation state",
  "status": 409,
  "detail": "Reservation '0ce386df-4dba-4290-b56c-6ba2831746ae' cannot transition from Confirmed to Confirmed.",
  "instance": "/api/v1/reservations/0ce386df-4dba-4290-b56c-6ba2831746ae/confirm",
  "errorCode": "INVALID_RESERVATION_STATE",
  "correlationId": "e0b13de4be9141dba06569b3aa28f8d0"
}
```

Download

Figure 8: Second confirm is rejected

Step 8. Check the product stock

Endpoint: GET /api/v1/products/{id}

- Paste the product `id` from Step 3 → **Execute**.

Expected: **200 OK** `totalQuantity: 3, reservedQuantity: 0, soldQuantity: 2, availableQuantity: 1.`



Figure 9: Stock after the sale

Step 9. Reserve and cancel

- POST /api/v1/reservations with { "productId": "<product id>", "quantity": 1 } → 201. Copy the new `reservationId`.
- POST /api/v1/reservations/{id}/cancel with that id → **Execute**.

Expected: **200 OK** `status: "Cancelled"` with `cancelledAtUtc` set. The unit is released (Step 8 again shows `availableQuantity: 1`).

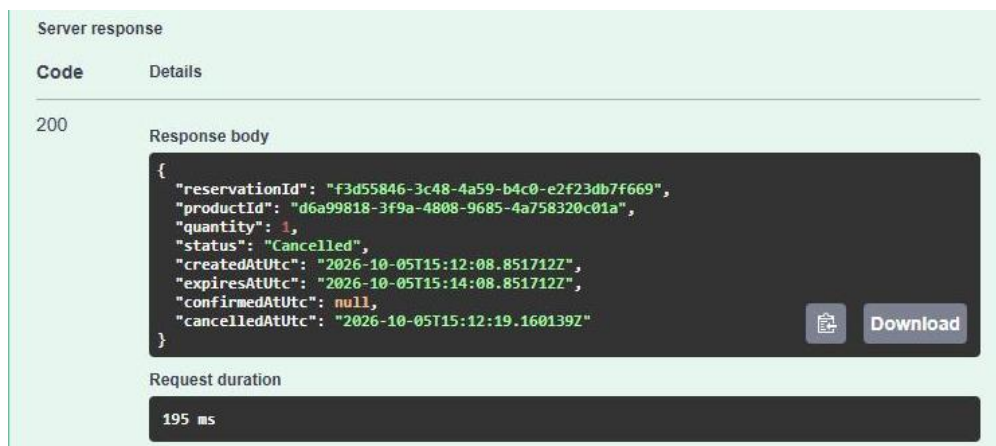


Figure 10: Reservation cancelled and stock released

Step 10. Automatic expiry after 2 minutes

- Reserve again: POST /api/v1/reservations with quantity: 1. Copy the `reservationId` and note `expiresAtUtc`.
- **Wait about 2 minutes 15 seconds** (the background worker sweeps every 15 seconds).
- Run GET /api/v1/products/{id}: `reservedQuantity` is back to 0 and `availableQuantity` to 1.
- Try to confirm the expired reservation: POST /api/v1/reservations/{id}/confirm.

Expected: **409 Conflict** `errorCode: "INVALID_RESERVATION_STATE"` ("cannot transition from Expired to Confirmed"). If you confirm after `expiresAtUtc` but before the sweep, you get `RESERVATION_EXPIRED` instead; either way the late purchase is refused.

Server response	
Code	Details
409	Error: Conflict
	Response body
	<pre>{ "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10", "title": "Invalid reservation state", "status": 409, "detail": "Reservation '7683a294-1514-44ef-b183-6da37dc4f5c7' cannot transition from Expired to Confirmed.", "instance": "/api/v1/reservations/7683a294-1514-44ef-b183-6da37dc4f5c7/confirm", "errorCode": "INVALID_RESERVATION_STATE", "correlationId": "68301a1df0ee49f88206760ed1e50248" }</pre>

Figure 11: An expired reservation cannot be confirmed

Step 11. Validation error

Endpoint: `POST /api/v1/reservations`

```
{
  "productId": "<product id>",
  "quantity": 0
}
```

Expected: **400 Bad Request** `errorCode: "VALIDATION_FAILED"` with a per-field message under `errors.quantity`.

Server response	
Code	Details
400	Error: Bad Request
	Response body
	<pre>{ "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1", "title": "One or more validation errors occurred.", "status": 400, "instance": "/api/v1/reservations", "errors": { "quantity": ["Quantity must be greater than zero."] }, "errorCode": "VALIDATION_FAILED", "correlationId": "1ab3b4d383934be3a3320886bbfac1f4" }</pre>

Figure 12: Validation error

Step 12. Call a protected endpoint without a token

- Click **Authorize** → **Logout** → **Close**. The padlocks open again.
- Run `GET /api/v1/products/{id}` again.

Expected: **401 Unauthorized** `errorCode: "UNAUTHORIZED"`, detail "A valid bearer token is required."
 Authorize again (Step 2) to continue.



Figure 13: Request without a token is rejected

Checklist

Every error response has the same shape (`type`, `title`, `status`, `detail`, `instance`, `errorCode`, `correlationId`), and every response carries an `X-Correlation-ID` header.

#	Action	Expected	Result
1	Login	200, JWT returned	☐
2	Authorize	Padlocks closed	☐
3	Create product (stock 3)	201, available 3	☐
4	Reserve 2	201, Active, expires +2 min	☐
5	Reserve 2 more	409 INSUFFICIENT_STOCK	☐
6	Confirm	200, Confirmed	☐
7	Confirm again	409 INVALID_RESERVATION_STATE	☐
8	Get product	200, reserved 0 / sold 2 / available 1	☐
9	Reserve 1 + cancel	200, Cancelled, stock released	☐
10	Reserve 1, wait 2 min, confirm	409, stock released by the worker	☐
11	Reserve quantity 0	400 VALIDATION_FAILED	☐
12	No token	401 UNAUTHORIZED	☐

Optional (role check): `POST /api/v1/products` with a token for a user whose role is `User` returns **403 FORBIDDEN**. Compose seeds only the admin; add a user with `SeedUsers__1__Username`, `SeedUsers__1__Password` and `SeedUsers__1__Role=User` in the `inventory-api` environment to try it. This case is covered by the automated tests.

7. How to test

7.1 Quick run (no database needed)

Runs all unit tests and all API-level integration tests; tests that need MySQL are reported as skipped.

```
dotnet test
```

Expected: **246 tests: 200 passed, 46 skipped, 0 failed** (about 30 seconds).

7.2 Full run including MySQL and the concurrency tests

Uses the MySQL container from docker-compose. Each test class creates and drops its own throwaway `inv_test_*` database, so application data is never touched.

```
docker compose up -d mysql

# bash / Git Bash / macOS / Linux
export INVENTORY_TEST_MYSQL=\
  "Server=localhost;Port=3306;User=root;Password=<MYSQL_ROOT_PASSWORD>"

# Windows PowerShell
$env:INVENTORY_TEST_MYSQL =
  "Server=localhost;Port=3306;User=root;Password=<MYSQL_ROOT_PASSWORD>"

dotnet test
```

Expected: **246 tests: 246 passed, 0 failed** (about 80 seconds). Use the `MYSQL_PORT` from `.env` if it is not 3306. The connection string has no database name; the user must be allowed to create databases (root).

7.3 Run only the concurrency tests (repeatable)

```
dotnet test tests/Inventory.IntegrationTests \
  --filter "FullyQualifiedName~Concurrent|FullyQualifiedName~Race"
```

7.4 What is covered

Area	Tests	Highlights
Unit: domain	invariants, transitions	Quantity rules, every allowed/forbidden reservation transition, terminal states, expiry boundary, user rules
Unit: application	services	Validation, ownership, outcome → error mapping, lost races, expiry batching, login (constant-time unknown user)
Integration: API	no database	Contracts, validation, 401/403, forged/expired tokens, ProblemDetails, correlation IDs, log hygiene, rate limits, OpenAPI document
Integration: MySQL	real database	Persistence, schema, migrations on startup, seeding, login, reserve/confirm/cancel/expiry, rollback on failed insert
Concurrency	real database	See table below

Concurrency scenario	Load	Verified result
Stock 1	500 simultaneous	1 success, 499 rejected, 0 oversold
Stock 100	500 simultaneous	100 success, 400 rejected, 0 oversold
Stock 100, full HTTP, 500 users, real rate limits	500 simultaneous	100 × 201, 400 × 409, 0 × 429
Concurrent confirm / concurrent cancel	20 on one reservation	Exactly one succeeds; sold/released once

Concurrency scenario	Load	Verified result
Confirm vs cancel	10 rounds × 20	Exactly one winner; stock matches the winner
Concurrent expiry	8 sweepers, 40 due	Each expired exactly once; no double release
Expiry vs confirm, expiry vs cancel	10 rounds each	Exactly one transition wins

The race tests were checked by mutation: removing the `Status = 'Active'` guard makes exactly those tests fail.

8. Suggested code review path

The most important logic is concentrated in a few files:

#	File	Why look at it
1	src/Inventory.Infrastructure/Persistence/Repositories/ReservationRepository.cs	Atomic conditional updates and transaction boundaries for reserve, confirm, cancel, expire
2	src/Inventory.Domain/Entities/Reservation.cs	State machine and the 2-minute rule
3	src/Inventory.Application/Reservations/ReservationService.cs	Use-case orchestration, validation, ownership
4	src/Inventory.Api/BackgroundJobs/ReservationExpiryWorker.cs	Periodic expiry, cancellation, graceful shutdown
5	src/Inventory.Infrastructure/Persistence/Configurations/	Schema: keys, CHECK constraints, indexes
6	src/Inventory.Api/Program.cs	Composition root and middleware order
7	tests/Inventory.IntegrationTests/Reservations/ReservationConcurrencyTests.cs	The 500-request concurrency verification
8	tests/Inventory.IntegrationTests/Reservations/ReservationLifecycleMySqlTests.cs	Confirm/cancel races

9. Configuration reference

Variable (.env)	Purpose	Required
MYSQL_ROOT_PASSWORD / MYSQL_PASSWORD	MySQL root and application user passwords	yes
MYSQL_DATABASE / MYSQL_USER / MYSQL_PORT	Database name, user, host port (defaults: inventory, inventory, 3306)	no
JWT_SIGNING_KEY	HMAC key for JWTs, at least 32 characters	yes

Variable (.env)	Purpose	Required
DEMO_ADMIN_USERNAME / DEMO_ADMIN_PASSWORD	Bootstrap admin created at startup (empty password = no seed)	no
API_PORT	Host port for the API (default 8080)	no
SWAGGER_ENABLED	Swagger UI on/off (default true for review)	no

Compose refuses to start if a required secret is missing. `.env` is excluded from source control and from the Docker build context; no secret is stored in the image.

10. Troubleshooting

Symptom	Fix
Set ... in <code>.env</code> error from docker compose	Create <code>.env</code> from <code>.env.example</code> and fill in the required values.
Port 3306 or 8080 already in use	Set <code>MYSQL_PORT</code> / <code>API_PORT</code> in <code>.env</code> , then <code>docker compose up -d</code> again.
Login returns 401 with the right password	The admin is created only once. If the volume already has that user with another password, run <code>docker compose down -v</code> and start again.
MySQL tests are all skipped	Set <code>INVENTORY_TEST_MYSQL</code> in the same terminal that runs <code>dotnet test</code> .
Concurrency tests time out on a slow machine	Give Docker Desktop more CPU/RAM and close other heavy workloads; the tests open up to ~500 connections.

11. Assumptions and known limitations

Assumptions

- Identifiers are GUIDs; one product per reservation; the reservation owner is the JWT `sub`.
- Users are provisioned through configuration (no self-registration).
- Expired-but-not-yet-swept reservations cannot be confirmed; their stock is released within one polling interval (15 s).
- API servers keep UTC clocks synchronized (NTP). TLS is terminated by a reverse proxy/ingress in production.

Known limitations and next steps

- **Hot products:** all reservations for one product queue on one row lock. For flash-sale traffic I would add fail-fast lock timeouts, per-product admission control, or split stock across several rows.
- **Idempotency:** add an `Idempotency-Key` header so client retries after a lost response cannot create duplicate holds.
- **Rate limits** are per instance and partitioned by remote IP. Behind a proxy, configure forwarded headers with known proxies and use a distributed store (e.g. Redis).

- **Expiry worker resilience:** isolate failures per reservation so one bad row cannot block later expiries; use **SKIP LOCKED** so instances split the work.
- **Retries:** retry MySQL deadlock/lock-timeout errors (1213/1205) for the whole transaction, since every operation is conditional.
- Production compose hardening: do not publish the MySQL port, disable Swagger, add HTTPS at the edge.

The repository README.md contains the full technical documentation, including architecture, schema and lifecycle diagrams.